OSS 1강 · 개요

Open Source Software 이해하기

저작권은 있지만 자유롭게 쓸 수 있는 소프트웨어 — 그 정의부터 참여 방법까지

목차

1 **용어와 정의**
Free Software와 OSS는 어떻게 다른가

2 **라이선스와 형태**
핵심 원칙과 대표 유형은 무엇인가

3 **장점과 단점**
왜 쓰고, 무엇을 조심해야 하는가

4 **역사와 중요성**
어떻게 성장해왔고, 왜 지금 중요한가

5 **생태계 참여**
GitHub에서 어떻게 기여를 시작하는가

용어 정리

Free Software

Free Software Foundation (FSF) · 윤리적 관점

사용자에게 실행·연구(수정)·배포·수정본 재배포라는 4대 자유를 보장하는 데 중점.
GPL 등 카피레프트 라이선스는 파생물도 동일한 자유를 유지하도록 요구.

Open Source Software

Open Source Initiative (OSI) · 실용적 관점

소스코드를 공개하는 것이 더 나은 소프트웨어를 만드는 방법이라는 관점.
공개된 코드에 대한 검증과 협업으로 품질과 개발 속도를 높이는 데 중점.

Proprietary Software

Closed Source Software

소스코드 비공개.
사용·수정·배포가 저작권자가 허락한 범위 안에서만 가능.

FOSS (Free Open Source Software), **FLOSS** (Free/Libre Open Source Software)
— 두 개념을 함께 아우르는 표현으로, 국내에서는 흔히 "공개 소프트웨어"라고 부릅니다.

OSS의 정의

저작권자가 소스코드를 공개하여,
누구나 자유롭게 사용·수정·재배포할 수 있는 SW

사용 수정 재배포

OSS에도 저작권은 존재합니다. "공개·협업"이 핵심이며, 라이선스가 정한 의무사항(출처 표기, 재배포 조건 등)은 반드시 지켜야 합니다.

→ 공유와 협업을 통한 지속적 발전

무료 배포되지만 소스가 비공개인 프로그램(예: 카카오톡)은 OSS가 아님 — 소스 공개도, 협업도 안 되니까

대표 라이선스

MIT	가장 느슨함 — 표시만 하면 자유롭게 사용·수정·재배포
Apache 2.0	MIT와 유사 + 특허 조항 포함
GPL	카피레프트 — 파생물도 동일 라이선스로 공개해야 함

Open Source Initiative(OSI)에는 약 80개의 인증 라이선스가 등록되어 있습니다.

OSS 라이선스, 핵심 원칙 5가지

OSI가 정한 10대 기준 중, 실무에서 가장 중요한 원칙만 정리했습니다

1

자유 재배포

로열티나 수수료 없이 누구나 재배포할 수 있어야 한다

2

소스코드 공개

소스코드를 함께 공개하여야 한다

3

파생 저작물 허용

수정을 통한 2차 저작물 창작을 허용해야 한다 (단, 라이선스에 따라 동일 조건 배포를 요구하기도 함)

4

무차별 원칙

특정 개인·단체·사용 분야·기술을 차별해서는 안 된다

5

라이선스 독립성

특정 제품에 종속되지 않고, 함께 배포되는 다른 SW를 제한하지 않는다

전체 10대 기준과 라이선스 전문(MIT, Apache, GPL 등)은 OSI 홈페이지(opensource.org)에서 확인할 수 있습니다.

OSS의 두 가지 형태

1) 패키지 형태 OSS

라이브러리·모듈 단위로 배포되며, 저작권·라이선스 고지문이 함께 포함됩니다.

```
Copyright (c) 1997-2000 Leon Brocard.
All rights reserved.

You may distribute this under the terms
of either the GNU General Public License
or the Artistic License.

This program is distributed WITHOUT ANY
WARRANTY; without even the implied
warranty of MERCHANTABILITY or FITNESS
FOR A PARTICULAR PURPOSE.
```

2) 프로젝트 형태 OSS

완성된 시스템 전체가 소스코드 단위로 공개되는 경우입니다.
대표적인 오픈소스 스택:

OS **Linux**	웹 서버 **Nginx, Apache**
DBMS **MySQL, PostgreSQL**	개발 언어 **Python, JavaScript**
프레임워크 **Django, Spring, React**	컨테이너/오케스트레이션 **Docker, Kubernetes**

패키지 형태는 "부품 단위"의 오픈소스(예: 라이브러리 하나), 프로젝트 형태는 "완제품 단위"의 오픈소스(예: 리눅스 운영체제 전체)

OSS의 장점

종속성 탈피

특정 벤더·기술의 Lock-in에서 벗어나고, 필요하면 직접 버그를 수정하고 요구사항 변화에 대응할 수 있습니다.

비용 절감

무료 라이선스를 통해 초기 도입 비용과 유지 비용을 크게 줄일 수 있습니다.

생산성 · 개발기간 단축

검증된 오픈소스 코드와 라이브러리를 재사용해 "바퀴를 다시 발명(reinvent the wheel)"하지 않아도 됩니다. 활성화된 커뮤니티가 신뢰성과 보안성도 함께 높여줍니다.

개발 역량 증대

공개된 소스코드로 알고리즘과 설계를 직접 분석·응용하며, 설계자·개발자·테스터로서의 기술력을 키울 수 있습니다.

OSS의 단점과 대응

품질 보증 부족

상용 제품과 달리 품질을 공식적으로 보증하는 기업이 없고, 프로젝트 지속성도 불확실할 수 있습니다.

대응 성숙한 커뮤니티와 유지관리 이력이 있는 프로젝트를 선택

IPR 관련 잠재 위협

소스 공개로 인한 영업비밀·핵심기술 유출 가능성, 라이선스 미준수 시 저작자와의 분쟁, 특허 침해 가능성이 있습니다.

대응 OSS에 대한 체계적인 관리 방안(Governance) 도입

OSS 역사, 자유SW에서 AI 시대까지

1983 — **GNU 프로젝트 시작**
리처드 스톨만, "Free Software" 개념 제시

1985 — **FSF 설립**
Free Software Foundation

1989 — **GNU GPL v1 발표**
리처드 스톨만, 최초의 카피레프트 라이선스

1991 — **Linux 커널 공개**
리누스 토발즈

1998 — **OSI 설립**
에릭 레이먼드, "Open Source" 용어 확산

1999 — **Apache 재단 설립**
Apache Software Foundation

2005 — **Git 최초 공개**
리누스 토발즈, 분산 버전관리 시스템

2007 — **GPL v3 발표**

2008 — **GitHub 창립**
협업 기반 오픈소스 확산의 전환점
(Pull Request 기반의 협업 방식)

2014 — **Docker 공개**
컨테이너 기술 대중화

2015~ — **AI 프레임워크 공개**
TensorFlow, PyTorch

2023~ — **오픈 웨이트 LLM 경쟁**
Llama, DeepSeek, Qwen ...

OSS의 중요성

기업 입장

- 생산성 증대, 개발 비용 절감
- No Vendor Lock-in
- OSS 기반의 자체 비즈니스 모델 가치 창출
  (예: Redhat : 오픈 소스 + 유료지원/컨설팅)

개발자 입장

- 코딩 능력 제고
- 협업 및 소통 능력 향상
- 개발 프로세스 전체 이해 — 설계 · 개발 · 테스트 · 배포 · 모니터링

취업 시, OSS 개발 이력(예: GitHub 계정)이 곧 가장 강력한 증명이 됩니다.

5 · 생태계 참여

OSS 생태계 참여하기

GitHub (github.com) — 리누스 토발즈가 만든 Git 기반의, 세계 최대 오픈소스 저장소입니다.

개발자라면 계정 필수!

Contributor → Committer, 참여의 흐름

찾아보기
관심 있는 프로젝트 탐색

→

살펴보기
코드와 이슈를 분석

→

문제 발견 · 제안
버그 리포트, 개선 아이디어

→

기여(PR)
코드 수정 및 Pull Request

→

리뷰 · 병합
Maintainer 리뷰 후 반영

꾸준히 기여하며 신뢰를 쌓으면 **Committer**(코드 리뷰·병합 권한을 가진 핵심 관리자)로 성장할 수 있습니다.

5 · 생태계 참여

OSS 프로젝트의 순환 구조

Core 팀과 커뮤니티가 서로 피드백을 주고받으며 프로젝트가 성장합니다

Core 팀 *핵심 개발자 · 관리자*

- 개발 · 문서화 · 테스트 담당
- 의사결정 — 설계 변경, 기능 추가 여부 결정

배포 단계

최신 스냅샷 → 베타(시험판) → 안정 릴리즈

Core 팀은 커뮤니티 기여를 검토해 다음 릴리즈에 반영할지 결정합니다.

배포 →

← Feedback

커뮤니티 *개발자 · 사용자*

- 기능 추가 제안 · 버그 수정 기여 (Pull Request)
- 버그 리포트 · 기능 요청 (Issue)

커뮤니티가 만들어내는 것

Patch(수정 소스) | Bug Report | Forum 논의

이 산출물이 다시 Core 팀에게 전달되며 순환이 이어집니다.

Maintainer가 기여를 검토·반영하는 이 순환이 반복되며 프로젝트가 발전합니다 — 체계적 관리(Governance)가 이 순환을 지속가능하게 만듭니다.

5 · 생태계 참여

오픈소스 개발의 실무 흐름

이슈 등록부터 배포까지 — ALM(Application Lifecycle Management)

1 **이슈 등록** 요구사항·버그 관리

→

2 **로컬 개발** 이슈 기반 코드 작성

→

3 **PR 제출** 변경사항 리뷰 요청

→

4 **코드 리뷰 · CI** 검토 + 자동 빌드/테스트

→

5 **품질 검사** 정적 분석 · 보안 스캔

→

6 **병합 · 배포** Merge → 자동 배포(CI/CD)

ALM(Application Lifecycle Management) 이란, 요구사항부터 배포까지 소프트웨어의 전체 생명주기를 관리하는 체계입니다.

오픈소스 프로젝트에서는 GitHub Issues, Pull Request 리뷰, GitHub Actions(CI/CD) 같은 도구들이 이 역할을 담당합니다.

이 강좌 2단계(GitHub Actions)·3단계(클라우드 배포)에서 위 흐름을 직접 실습합니다.

정리하며

OSS는 “공짜 코드”가 아니라,
저작권 위에서 자유와 책임을 함께 나누는 협업의 규칙입니다.

참고문헌

- 「공개 SW 라이선스 가이드」, 정보통신산업진흥원(NIPA)
- 「Open Source Software and GitHub」, 이민석, 국민대학교
- 「기업이 알아야 할 오픈소스 라이선스」, Black Duck Software Korea
- Open Source Initiative, opensource.org